

How a Neural Network Learns

From a spam filter to ChatGPT, in fifteen minutes

Sathish Gopalakrishnan

The University of British Columbia

Write Me a Spam Filter

The way you were taught

```
if "free" in subject:
    spam = True
if links > 5:
    spam = True
if sender in contacts:
    spam = False
...
```

You write the rules.

Key Takeaway

Today: the smallest working version of that idea, in Python you could read in a minute. Then how it scales to images, then to language.

The other way

Show the program a pile of emails, each labelled *spam* or *fine*, and let it work out the rules itself.

That second idea — and almost nothing else — is what got us image recognition, speech recognition, and ChatGPT.

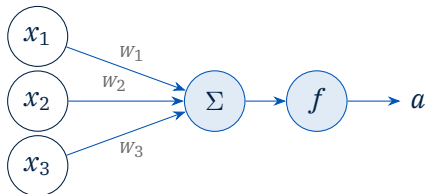
One Neuron

Weigh, add, decide

The Whole Building Block

A **neuron** weighs its inputs — a few numbers called **features** — adds a bias, and passes the total through a simple function f :

$$a = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$



Multiplication and addition. The only wrinkle is f — here **ReLU**: keep the total if it is positive, otherwise output zero.

Two Emails, Three Numbers

From: promo@dealz4u.biz
Subject: your **FREE** iPhone is waiting

You have been selected! Claim the **free** gift before midnight:

bit.ly/a4z9 bit.ly/kk2p
bit.ly/9qmx tinyurl.com/w7

links	4
says “free”	1
sender in contacts	0

$$\mathbf{x}_A = (4, 1, 0)$$

From: priya@ubc.ca
Subject: notes from today’s lab

Here is what we wrote down before the TA kicked us out:

ubc.ca/cpen211/lab3

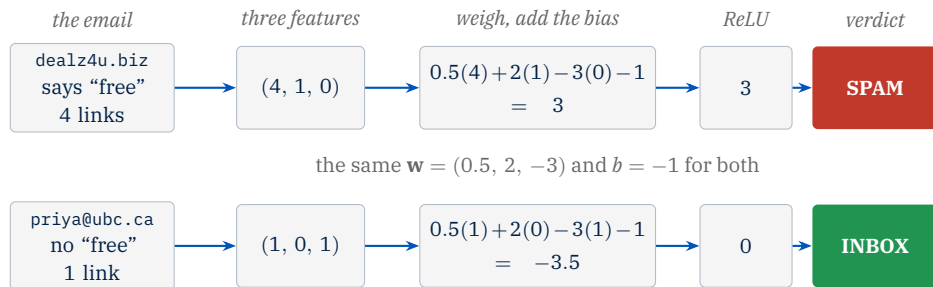
See you Thursday.

links	1
says “free”	0
sender in contacts	1

$$\mathbf{x}_B = (1, 0, 1)$$

Each number is a **feature**: one thing about the email we chose to measure. Somebody had to decide that *these* three are the features worth counting. Hold that thought.

Running the Filter



Two emails, the same four numbers, opposite answers. The neuron never reads a word: by the time it runs, an email *is* its three features.

That Neuron, in Python

```
def relu(z):  
    return max(0.0, z)  
  
def neuron(x, w, b):  
    z = sum(wi * xi  
            for wi, xi in zip(w, x)) + b  
    return relu(z)  
  
w = [0.5, 2.0, -3.0]  
b = -1.0  
  
neuron([4, 1, 0], w, b)    # 3.0  -> spam  
neuron([1, 0, 1], w, b)    # 0.0  -> fine
```

No libraries. No GPU. Twelve lines.

Every neuron in every model you have heard of does *this*, with more inputs.

One problem: I chose those three weights by hand.

Same Neuron, Steering a Car

The same neuron can keep a car in its lane, and here the answer is an angle rather than a yes or no. Right is positive throughout.

	offset m	heading degrees	bend ahead degrees	<i>weigh, add $b = 1$</i>	<i>clip at ± 30</i>	<i>steer</i>
weight w	-10	-1	0.5			
Car A	0.5	4	0	$-5 - 4 + 0 + 1 = -8$	-8	8° LEFT
Car B	0	0	20	$0 + 0 + 10 + 1 = 11$	11	11° RIGHT
Car C	2	15	0	$-20 - 15 + 0 + 1 = -34$	-30	30° LEFT

```
def steer(x, w, b):  
    z = sum(wi * xi  
            for wi, xi in zip(w, x)) + b  
    return max(-30, min(30, z))
```

Only the last line changed. ReLU clips at zero; a wheel clips at $\pm 30^\circ$. f is whatever suits the answer, and the next section uses a third one.

These weights are controller gains, and I still picked them by hand.

Learning

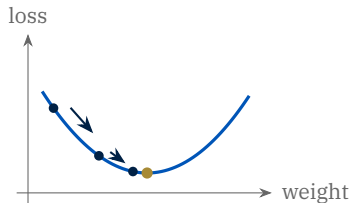
We optimise the weights against data

Learning Is Just Rolling Downhill

We **optimise** the weights against emails we have already labelled:

1. Start with weights of **zero** — a model that knows nothing.
2. For each email, compare the guess to the truth. The gap is the **error**.
3. **Nudge** every weight a little in the direction that shrinks that error.
4. Repeat a few hundred times.

The average error over all the data is called the **loss**. Training = pushing the loss down.



each step moves downhill

Training, in Python

```
DATA = [( [4,1,0], 1), ([6,1,0], 1), ([3,1,1], 1), ([5,0,0], 1), ([2,1,0], 1),  
         ([1,0,1], 0), ([0,0,1], 0), ([2,0,1], 0), ([1,0,0], 0), ([0,1,1], 0)]
```

```
def train(data, steps=100, lr=0.1):  
    w, b = [0.0, 0.0, 0.0], 0.0           # know nothing  
    for step in range(steps):  
        for x, y in data:  
            error = predict(x, w, b) - y   # how wrong were we?  
            for i in range(len(w)):  
                w[i] -= lr * error * x[i]   # nudge each weight downhill  
            b -= lr * error  
    return w, b
```

Ten emails, four numbers to learn, one loop. `predict` is the same neuron — with a smooth f , so the output is a *probability*.

What the Optimisation Produced

training:

step	0	loss	0.693
step	25	loss	0.137
step	50	loss	0.084
step	75	loss	0.061
final		loss	0.048

what it learned:

links	+1.89
'free'	+2.61
known sender	-2.45
bias	-3.62

We never typed +2.61. We optimised it out of ten labelled emails.

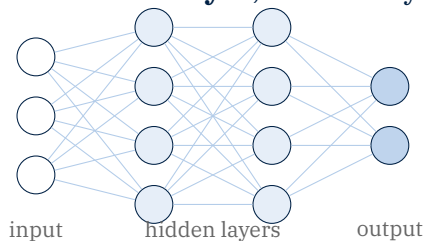
And look at the *signs*: links and “free” positive, a known sender negative. Exactly the reasoning a person would use, recovered from the data by arithmetic.

On an email it had never seen,
[3, 1, 0]:

0.99 probability of spam

From One Neuron to a Deep Network

Neurons side by side make a **layer**; stacked layers make it **deep**.



Every arrow is a weight, and every weight is learned by the loop you just read.

Key Takeaway

Going deep adds no new idea — just more of the same arithmetic. **Deep learning** is this, at scale.

Images

When the network picks the features

Images Break the Trick

For email, *I* chose the features; for the car, an engineer did. For a photo, what would you choose?

The obvious idea: one input per pixel.

A 200×200 colour photo is **120,000** numbers. Pixel #8,412 means nothing on its own.

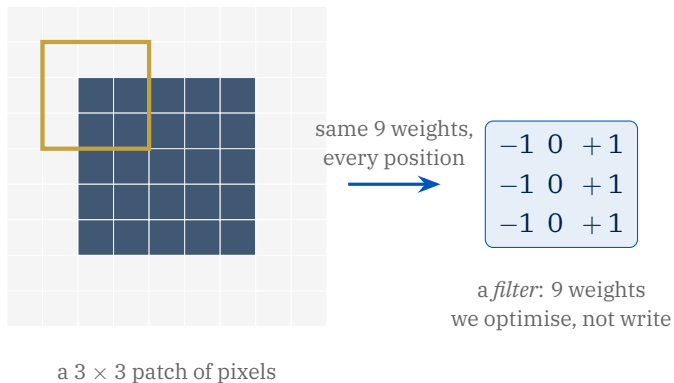
And it is fragile: shift the cat three pixels to the right and every single input changes.

A cat is a cat wherever it sits in the frame.

So: stop looking at the whole image at once.

Look at a **small patch**, and look everywhere.

A Convolution Is One Neuron, Slid Around



Multiply the 9 pixels by the 9 weights, add, ReLU — the same neuron as before. Slide it everywhere and you get a map of *where that pattern occurs*: a **feature** the network made, not one we picked.

Run It Over a Real Picture

```
for r in range(len(image) - 2):           # every position, top to bottom
    for c in range(len(image[0]) - 2):     # every position, left to right
        total = sum(image[r+i][c+j] * kernel[i][j])
            for i in range(3) for j in range(3)
        out[r][c] = max(0, total)          # ReLU, same as before
```

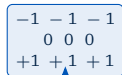


a 28×28 image
(784 numbers)

run
through



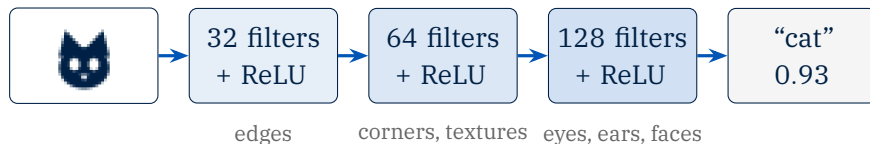
finds **left** edges



finds **top** edges

Here the nine numbers are mine. A real CNN optimises them against labelled images.

Stack Them, and Features Build Themselves



Layer 1's outputs are layer 2's features. Edges combine into corners, corners into eyes, eyes into faces — and *we programmed none of those features*.

Key Takeaway

This is a **CNN**. Same neuron, same downhill nudge — roughly a million weights instead of four, optimised against tens of thousands of labelled images.

What This Looks Like in Practice

```
model = nn.Sequential(  
    nn.Conv2d(1, 32, 3), nn.ReLU(), nn.MaxPool2d(2),    # find edges  
    nn.Conv2d(32, 64, 3), nn.ReLU(), nn.MaxPool2d(2), # combine them  
    nn.Flatten(),  
    nn.Linear(64 * 5 * 5, 10),                        # 10 possible answers  
)  
  
for images, labels in loader:  
    loss = loss_fn(model(images), labels)    # how wrong were we?  
    loss.backward()                          # which way is downhill?  
    optimizer.step()                        # nudge every weight  
    optimizer.zero_grad()
```

The library writes the loop for you and runs it on a GPU. It is still *your* loop: guess, measure the error, nudge.

Language

The same recipe, one more time

An LLM Is This, With a Different Question

Spam filter

Input: 3 features, chosen by hand

Question: *spam or not?*

Data: 10 labelled emails

Optimised: 4 weights

CNN

Input: pixels; the features it builds itself

Question: *which of 10 things is this?*

Data: 50,000 labelled images

Optimised: $\sim 10^6$ weights

Large language model

Input: the text so far, as numbers

Question: *what word comes next?*

Data: much of the public internet

Optimised: 10^{11} – 10^{12} weights

No human labelled any of it: the next word in the sentence *is* the label. That is why there was so much training data lying around.

Key Takeaway

Guess, measure the error, nudge. Same loop you read three slides ago.

One Genuinely New Idea, and Then Scale

The new idea: attention. Before each layer does its arithmetic, every word looks at every other word and pulls in what it needs.

*The animal crossed the street because **it** was tired.*

“it” asks the sentence *who am I?* — and mostly gets back “animal.”

Then: scale. About a hundred such layers, trained for months on thousands of GPUs, at a reported cost of tens of millions of dollars. Somewhere in there it starts writing code and translating Latin — neither of which anybody put in as a rule.

Key Takeaway

Nobody fully understands why so much capability falls out of “predict the next word.” That question is open, and a good reason to work on this.

What to Take Away

1. A **neuron** weighs its features, adds, and clips. That is all it does.
2. **Learning** is: guess, measure how wrong you are, nudge every weight downhill. Repeat.
3. This time we picked the **features** by hand. The weights we **optimise** against data, which recovered the signs a human would have picked.
4. **Deep** means stacking layers so that the features build themselves: edges → eyes → faces. Nobody picks them.
5. An **LLM** is the same loop, asking “what comes next?”, with a hundred billion weights instead of four.

Every line of code in this talk was plain Python.

What separates it from a frontier model is scale.